LAB MANUAL

On

NEURAL NETWORKS AND DEEP LEARNING LAB

COURSE CODE (22CDPC55)

(IV- B. Tech. – I– Semester)

Submitted to

DEPARTMENT OF COMPUTER SCIENCE& ENGINEERING (AIML) and AIML

By

Mrs Shaik.Jasmine

(Asst. Professor, Dept. of AI & ML)

CMR INSTITUTE OF TECHNOLOGY

Kandlakoya(V), Medchal Road, Hyderabad – 501 401

Ph. No. 08418-222042, 22106 Fax No. 08418-222106

(2025-26)

CONTENTS

CMR INSTITUTE OF TECHNOLOGY

VISION: To create world class technocrats for societal needs

MISSION: Achieve global quality technical education by assessing learning environment through

- Innovative Research & Development
- Eco_system for better industry institute interaction
- capacity building among stake holders

QUALITY POLICY: Strive for global excellence in academics and research to the satisfaction of students and stakeholders

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING:
COMPUTER SCIENCE AND ENGINEERING (ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING)

Vision: Develop competent software professionals, researchers and entrepreneurs to serve global socity

Mission: The department of Computer science and engineering (AIML) is committed to

- create technocrats with proficiency in design and code for software development
- adapt contemporary technologies by lifelong learning and face challenges in IT and ITES Sectors

I. PROGRAMME EDUCATIONAL OBJECTIVES (PEO's) : Engineering Graduates Will

PEO1: Pursue successful professional career in IT and IT-enabled sectors

PEO2: pursue lifelong learning skills to solve complex problems through multidisciplinary-research.

PEO3: Exhibites professionalism, ethics and inter-personal skill to develop leader ship qualities

II. PROGRAMME OUTCOMES (PO's)

1. **Engineering knowledge**: Apply mathematics, science, engineering fundamentals, to solve complex engineering problems. **[PEO's: 1,2 and 3]**

2. **Problem analysis**: Identify, formulate and analyze complex engineering problems to reach substantiated conclusions **[PEO's: 1,2 and 3]**

3. **Design/development of solutions**: Design and develop a components /system/process to slove complex societal engineering problems**[PEO's: 1,2 and 3]**

4. **Conduct investigations of complex problems**: Design and conduct experiments to analyze, interpret and synthesize data for valid conclusion. **[PEO's: 1,2 and 3]**

5. **Modern tool usage**: Create, select, and apply modern tools, resources to slove complex engineering problems. **[PEO's: 1,2 and 3]**

6. **The engineer and society:** Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice. **[PEO's: 2 and 3]**

7. **Environment and sustainability**: Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development. **[PEO's: 1,2 and 3]**

8. **Ethics:** Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice. **[PEO's: 1,2 and 3]**

9. **Individual and team work:** Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings. **[PEO's: 1,2 and 3]**

10. **Communication:** Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions. **[PEO's: 1,2 and 3]**

11. **Project management and finance**: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments. **[PEO's: 1 and 3]**

12. **Life-long learning:** Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change. **[PEO's: 1,2 and 3]**

NEURAL NETWORKS AND DEEP LEARNING LAB

Course B.Tech.-VII-Sem. **L T P C**
Subject Code 22CAPC72 **- - 2 1**

Course Outcomes
Upon completion of course the students will be able to

1. develop programs for neural networks
2. implement perceptron learning for linearly separable problem
3. sketch multiple curve using neural networks programs
4. use Spyder IDE Environment for Python Programs
5. apply the Autoencoder algorithms for encoding the real-world data

LISTOFEXPERIMENTS

1. Perform an experiment to show how the weight and bias value effects the value of neurons.
2. Perform an experiment to show how the choice of activation function effect the output of neuron experiment with purelin(n).
3. Perform an experiment to show how the choice of activation function effect the output of neuron experiment with Tansig(n).
4. Perform an experiment to show how the choice of activation function effect the output of neuron experiment with logsig(n).
5. Perform an experiment to show how the perceptron learning rule works for linearly separable problems.
6. Perform an experiment to show how the perceptron learning rule works for Non-linearly separable problems.
7. Write a program to draw a graph with multiple curve.
8. Setting up the Spyder IDE Environment and Executing a Python Program.
9. Installing Keras, Tensorflow and Pytorch libraries and making use of them.
10. Train a sentiment analysis model on IMDB dataset, use RNN layers with LSTM/GRU notes.
11. Applying the Autoencoder algorithms for encoding the real-world data.
12. Applying Generative Adversial Networks for image generation and unsupervised tasks.

Micro-Projects:

1. Build an API for implementing Multilayer perceptron for XOR problem.
2. Build a slack Boat for file uploading.
3. Build an API to implement Back Propagation in Neural Networks.
4. Build a simple proxy server.
5. Build an Ecommerce website
6. Build a convolutional neural network by implementing convolutional function.
7. Build a CRUD API using go programming.
8. Build a Multilayer perceptron neural network for classification.
9. Build an API to construct a Neural network to implement Linearly separable problem.
10. Build an API to construct a Neural network to implement feature extraction.

References :

Neural Networks and Deep Learning Lab Manual, Department of AI&ML, CMRIT, Hyd.

1. STUDENT ENTRY BEHAVIOR OR PRE-REQUISITES

- Students should have basic knowledge on statistics and programming (python)
- Students should have basic knowledge build a foundation in mathematics (linear algebra,calculus)
- Student should have knowledge on Python.

These prerequisites are taken by the students during the first two years. However during the initial sessions the topics are reviewed.

2. COURSE OBJECTIVES

Course Objectives	Course Objective Statements
Objective - 1	Explain the concepts of neural network and deep learning
Objective – 2	Adapt various deep learning algorithms and the way to evaluate performance of the deep learning algorithms.
Objective – 3	To apply Deep learning to learn ,predict and classify the real-world problems
Objective – 4	To understand the concept of machine learning
Objective – 5	Perceive various reinforcement learning approaches

3. COURSE OUTCOMES

COs	Uponcompletionofcoursethestudents willbeableto	PO4	PO5	PO9	PSO2
CO1	develop programs for neural networks	3	3	3	3
CO2	implement perceptron learning for linearly separable problem	3	3	3	3
CO3	sketch multiple curve using neural networks programs	3	3	3	3
CO4	use Spyder IDE Environment for Python Programs	3	3	3	3
CO5	apply the Autoencoder algorithms for encoding the real-world data	3	3	3	3

Course Mapping

Course Name	PEO1	PEO2	PEO3	PSO1	PSO2	PSO3
NEURAL NETWORKS AND DEEP LEARNING LAB					√	

Course Name	PO_1	PO_2	PO_3	PO_4	PO_5	PO_6	PO_7	PO_8	PO_9	PO_{10}	PO_{11}	PO_{12}
NEURAL NETWORKS AND DEEP LEARNING LAB				√	√				√			

4. MAPPING OF COURSE WITH PEOS-PSOS-POS

Program Educational Objectives (PEOs)

Sl. No.	PEOs Name	Program Education Objective Statements
1	PEO - 1	Pursue successful professional career in IT and IT-enabled sectors [PO's: 1,2,3,4,5,7,8,9,10,11 and 12] [PSO's: 1 and 2]
2	PEO – 2	pursue lifelong learning skills to solve complex problems through multidisciplinary-research. [PO's: 1,2,3,4,5,6,7,8,9,10 and 12] [PSO's: 1, 2 and 3]
3	PEO – 3	Exhibites professionalism, ethics and inter-personal skill to develop leader ship qualities [PO's: 1,2,3,4,5,6,7,8,9,10,11 and 12] [PSO's: 2 and 3]

Program Specific Objectives (PSOs)

Sl. No.	PSOs Name	Program Specific Objective Statements
1	PSO - 1	Design and develop computer - based-system using Algorithm, Networks,Security,Gaming, Full Stack,Devops.IoT,Cloud,Data Science and AI&ML.[PO:1,2,3,4 and 5] & [PEO:1 and 2]
2	PSO – 2	Apply techniques of AIML to solve real world problems .[PO:1,2,3,4,5,10 and 11] & [PEO:1,2 and 3]
3	PSO – 3	Ensure employability and career development skills through Industry oriented mini & major projects, internship, industry visits, seminars and workshops. [PO:6,7,8,9,10,11 and 12] & [PEO:1,2 and 3]

Program Outcomes (POs)

PO Name	Graduate Attributes	PO Statements
PO1	Engineering knowledge	Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems. [PEO's: 1,2 and 3] [PSO's: 1,2 and 3]
PO 2	Problem analysis	Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences. [PEO's: 1,2 and 3] [PSO's: 1,2 and 3]
PO 3	Design/ development of solutions	Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations. [PEO's: 1,2 and 3] [PSO's: 1,2 and 3]
PO 4	Conduct investigations of complex problems	Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions. [PEO's: 1,2 and 3] [PSO's: 1,2 and 3]
PO 5	Modern tool usage	Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations. [PEO's: 1,2 and 3] [PSO's: 1,2 and 3]
PO 6	The engineer and society	Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice. [PEO's: 2 and 3]
PO 7	Environment and sustainability	Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development. [PEO's: 1,2 and 3]
PO 8	Ethics	Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice. [PEO's: 1,2 and 3] [PSO's: 2 and 3]
PO 9	Individual and team work	Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings. [PEO's: 1,2 and 3] [PSO's: 3]
PO 10	Communication	Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions. [PEO's: 1,2 and 3] [PSO's: 2 and 3]
PO 11	Project management and finance	Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments. [PEO's: 1 and 3] [PSO's: 2 and 3]
PO 12	Life-long learning	Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change. [PEO's: 1,2 and 3] [PSO's: 1,2 and 3]

MAPPINGOFCOURSEOUTCOMESWITHPEO'S,PO'S
(Nocorrelation:0;Low:1;Medium:2;High:3)

Course Outcomes	PEO1	PEO2	PEO3	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12
CO-1							3	3				3			
CO–2							3	3				3			
CO–3							3	3				3			
CO–4							3	3				3			
CO–5							3	3				3			

5. Mapping Of Course Outcomes With PEOs

(Ticking & Correlation - 2 Tables)

No	Course Outcomes	PEO1	PEO2	PEO3
1	CO - 1			
2	CO – 2			
3	CO – 3			
4	CO – 4			
5	CO – 5			

6. Mapping Of Course Outcomes With PSOs

(Ticking & Correlation - 2 Tables)

No	Course Outcomes	PSO1	PSO2	PSO3
1	CO - 1		√	
2	CO – 2		√	
3	CO – 3		√	
4	CO – 4		√	
5	CO – 5		√	

No	Course Outcomes	PSO1	PSO2	PSO3	Avg
1	CO - 1				
2	CO – 2				
3	CO – 3				
4	CO – 4				
5	CO – 5				
	Avg				

7. Mapping Of Course Outcomes With POs

(Ticking & Correlation - 2 Tables)

No	Course Outcomes	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12
1	CO - 1				√	√				√			
2	CO – 2				√	√				√			
3	CO – 3				√	√				√			
4	CO – 4				√	√				√			
5	CO – 5				√	√				√			

No	Course Outcomes	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12	Avg
1	CO - 1													
2	CO – 2													
3	CO – 3													
4	CO – 4													
5	CO – 5													
	Avg													

8. Direct Course Assessment

(As mentioned in following table of 10 parameters, of which consider only the parameters required for this courses)

No	Description	Targeted Performance	Actual Performance	Remarks	Course Attainment
1	Internal Marks(25)	80% of Students(182 Students) should Secure 60% of Internal Marks i.e., 15 Marks			
2	External Marks(50)	80% of Students(182 Students) should Secure 70% of External Marks i.e., 35 Marks			
3	Clearing of Subject	A minimum of 95% of Students(216 Students) should clear this course in first attempt			
4	Getting First Class	90% of Students(205 Students) should Secure I Class Marks i.e., 45 Marks in my course			
5	Distinction	80% of Students (182 Students) should secure First Class With Distinction i.e., 53 Marks in my course			
6	Outstanding Performance	60% of Students (137 Students) should secure 80% and above Marks. i.e., 60 Marks in my course			

9. Indirect Course Assessment

(As mentioned-strong (3), moderate (2), weak (1) & no comment (0))

Mission Statement of CSE

Impart fundamentals through state of art technologies for research and career in Computer Science & Engineering.

Create value-based, socially committed professionals for anticipating and satisfyingfast changing societal requirements.

Foster continuous self learning abilities through regular interaction with various stake holders for holistic development

Correlation of Mission Elements with Mission Statement of CSE Department related to the Course (only Ticking given by faculty)

No	Mission Elements	Strong	Moderate	Weak	No Comment
M-1	Impart Fundamentals	√			
M-2	State Of Art Technologies	√			
M-3	Research & Career Development	√			
M-4	Value based Socially Committed Professional	√			
M-5	Anticipating & Satisfying Industry Trends		√		
M-6	Changing Societal Requirements			√	
M-7	Foster Continuous Learning	√			
M-8	Self Learning Abilities	√			
M-9	Interaction with stakeholders	√			
M-10	Holistic Development		√		

Indirect Course Assessment through Student Satisfaction Survey

(Note for *:Parameters used for course teaching like

a: Classroom teaching b: Simulations c:labs d: Mini_Projects
e: Major Projects f: Conferences g: professional activities
h: Technical Clubs i: Guest Lectures j: Workshops k: Technical Fests
l:Tutorials m:NPTLs n:Digital Library o: Industrial Visits
p: software Tools q: Internship/training r:Technical Seminars
s: NSS t: NSS u: sports etc.

No	Question Based on PEO/ PO/PSO/CO	Parameters (a /b /c…/)*	Strong (3)	Moderate (2)	Weak (1)	No comment (0)
1	Did the course impart fundamentals through interactive learning and contribute to core competence?					
2	Did the course provide the required knowledge to foster continuous learning?					
3	Whether the syllabus content anticipates & satisfies the industry and societal needs?					
4	Whether the course focuses on value based education to be a socially committed professional?					
5	Rate the role of the facilitator in mentoring and promoting the self learning abilities to excel academically and professionally?					
6	Rate the methodology adopted and techniques used in teaching learning processes?					
7	Rate the course in applying sciences & engineering fundamentals in providing research based conclusions with the help of modern tools?					
8	Did the course have any scope to design, develop and test a system or component?					
9	Rate the scope of this course in addressing cultural, legal, health, environment and safety issues?					
10	Scope of applying management fundamentals to demonstrate effective technical project presentations & report writing?					
	Total					
	Average					
Total Average			2.52			

10. Overall Course Assessment

(80% Direct + 20% Indirect, if any)

No	Assessment Type	Weightage	Attainment Level
1	Direct-Assignment, Quiz, Subjective, University Exams, Results, Bench Marks	0.8	
2	Indirect-Surveys-Questionnaire	0.2	
	Overall		

AI & ML LAB Course Attainment level:

11. Pi diagrams, Bar charts, Histograms

(For representing previous results, if any)

WT Pass % for Last 4 Academic Years	Appeared	Passed	Pass%

INDEX

EXPERIMENT NO: 1

AIM : Perform an experiment to show how the weight and bias value effects the value of neurons.

ALGORITHM:

SOURCE CODE:

```
import numpy as np

class NeuralNetwork:
    def __init__(self, input_size, hidden_size, output_size):
        # Initialize the weights and biases for the layers

        # Input to hidden layer weights (random initialization)
        self.weights_input_hidden = np.random.randn(input_size, hidden_size)
        # Hidden layer biases
        self.bias_hidden = np.zeros((1, hidden_size))

        # Hidden to output layer weights (random initialization)
        self.weights_hidden_output = np.random.randn(hidden_size, output_size)
        # Output layer biases
        self.bias_output = np.zeros((1, output_size))

        # Print the initialized weights and biases
        print("Initialized weights and biases:")
        print("Weights (Input -> Hidden):\n", self.weights_input_hidden)
        print("Biases (Hidden Layer):\n", self.bias_hidden)
        print("Weights (Hidden -> Output):\n", self.weights_hidden_output)
        print("Biases (Output Layer):\n", self.bias_output)

    def forward(self, X):
        # Forward pass through the network

        # Input to hidden layer activation (using sigmoid for example)
        self.hidden_input = np.dot(X, self.weights_input_hidden) + self.bias_hidden
        self.hidden_output = self.sigmoid(self.hidden_input)

        # Hidden to output layer activation
        self.output_input = np.dot(self.hidden_output, self.weights_hidden_output) + self.bias_output
        self.output = self.sigmoid(self.output_input)

        return self.output

    def sigmoid(self, x):
```

```
        return 1 / (1 + np.exp(-x))

# Example usage:
input_size = 3    # Number of input neurons
hidden_size = 5   # Number of neurons in the hidden layer
output_size = 2   # Number of output neurons

# Create the neural network instance
nn = NeuralNetwork(input_size, hidden_size, output_size)

# Example input data (for a batch of 2 samples with 3 features each)
X_input = np.array([[0.1, 0.5, 0.9], [0.2, 0.8, 0.6]])

# Perform a forward pass with the input data
output = nn.forward(X_input)
```

OUTPUT:

1. Initialized weights and biases:

```
 Weights (Input -> Hidden):
[[ 0.13733665  0.60730689 -0.93537005 -1.47362686 -1.37556107]
 [ 0.03513254  0.50556858 -1.04688382 -1.22800211  0.09775871]
[-1.37266931  1.2456073   1.73657668  0.13192531 -0.48666199]]
 Biases (Hidden Layer):
 [[0. 0. 0. 0. 0.]]
Weights (Hidden -> Output):
[[ 1.28871665 -0.53919145]
[ 1.33294701  0.9834711 ]
[-2.16332141 -1.31205913]
[ 0.71992958 -0.4048897 ]
[-1.21746859  0.14903262]]
Biases (Output Layer): [[0. 0.]]
Output of the neural network for the input data:
 [[0.40413155 0.41109537]
 [0.51571203 0.47466632]]
```

EXPERIMENT NO: 2

AIM : Perform an experiment to show how the choice of activation function effect the output of neuron experiment with purelin(n).

SOURCE CODE:

```
import numpy as np

class NeuralNetwork:
    def __init__(self, input_size, hidden_size, output_size, activation_function='sigmoid'):
        # Initialize the weights and biases
        self.weights_input_hidden = np.random.randn(input_size, hidden_size)
        self.bias_hidden = np.zeros((1, hidden_size))

        self.weights_hidden_output = np.random.randn(hidden_size, output_size)
        self.bias_output = np.zeros((1, output_size))

        # Set the activation function
        self.activation_function = activation_function

    def forward(self, X):
        # Forward pass through the network
        self.hidden_input = np.dot(X, self.weights_input_hidden) + self.bias_hidden
        self.hidden_output = self.apply_activation(self.hidden_input)

        self.output_input = np.dot(self.hidden_output, self.weights_hidden_output) + self.bias_output
        self.output = self.apply_activation(self.output_input)

        return self.output

    def apply_activation(self, x):
        if self.activation_function == 'sigmoid':
            return self.sigmoid(x)
        elif self.activation_function == 'relu':
            return self.relu(x)
        elif self.activation_function == 'tanh':
            return self.tanh(x)
        elif self.activation_function == 'leaky_relu':
            return self.leaky_relu(x)
        elif self.activation_function == 'softmax':
            return self.softmax(x)
```

```
        elif self.activation_function == 'softplus':
            return self.softplus(x)
        else:
            raise ValueError(f"Activation function {self.activation_function} not supported.")

    # Activation functions
    def sigmoid(self, x):
        return 1 / (1 + np.exp(-x))

    def relu(self, x):
        return np.maximum(0, x)

    def tanh(self, x):
        return np.tanh(x)

    def leaky_relu(self, x, alpha=0.01):
        return np.where(x > 0, x, alpha * x)

    def softmax(self, x):
        e_x = np.exp(x - np.max(x, axis=-1, keepdims=True))  # Numerical stability
        return e_x / np.sum(e_x, axis=-1, keepdims=True)

    def softplus(self, x):
        return np.log(1 + np.exp(x))

# Example usage:
input_size = 3    # Number of input neurons
hidden_size = 5   # Number of neurons in the hidden layer
output_size = 2   # Number of output neurons

# Create a neural network instance
nn = NeuralNetwork(input_size, hidden_size, output_size, activation_function='sigmoid')

# Example input data (for a batch of 2 samples with 3 features each)
X_input = np.array([[0.1, 0.5, 0.9], [0.2, 0.8, 0.6]])

# Perform a forward pass with the input data
output = nn.forward(X_input)

print("\nOutput of the neural network for the input data:")
print(output)
```

Output:

Output of the neural network for the input data:

```
[[0.6854294  0.35951906]
[0.70518654 0.37606834]]
```

EXPERIMENT NO: 3

AIM: Perform an experiment to show how the choice of activation function effect the output of neuron experiment with Tansig(n).

ALGORITHM

SOURCE CODE:

```
import numpy as np

class NeuralNetwork:
    def __init__(self, input_size, hidden_size, output_size, activation_function='sigmoid',
loss_function='mse'):
        # Initialize the weights and biases
        self.weights_input_hidden = np.random.randn(input_size, hidden_size)
        self.bias_hidden = np.zeros((1, hidden_size))

        self.weights_hidden_output = np.random.randn(hidden_size, output_size)
        self.bias_output = np.zeros((1, output_size))

        # Set the activation function
        self.activation_function = activation_function
        self.loss_function = loss_function

    def forward(self, X):
        # Forward pass through the network
        self.hidden_input = np.dot(X, self.weights_input_hidden) + self.bias_hidden
        self.hidden_output = self.apply_activation(self.hidden_input)

        self.output_input = np.dot(self.hidden_output, self.weights_hidden_output) + self.bias_output
        self.output = self.apply_activation(self.output_input)

        return self.output

    def apply_activation(self, x):
        if self.activation_function == 'sigmoid':
            return self.sigmoid(x)
        elif self.activation_function == 'relu':
            return self.relu(x)
        elif self.activation_function == 'tanh':
            return self.tanh(x)
        elif self.activation_function == 'leaky_relu':
            return self.leaky_relu(x)
        elif self.activation_function == 'softmax':
            return self.softmax(x)
```

```
        elif self.activation_function == 'softplus':
            return self.softplus(x)
        else:
            raise ValueError(f"Activation function {self.activation_function} not supported.")

    # Activation functions
    def sigmoid(self, x):
        return 1 / (1 + np.exp(-x))

    def relu(self, x):
        return np.maximum(0, x)

    def tanh(self, x):
        return np.tanh(x)

    def leaky_relu(self, x, alpha=0.01):
        return np.where(x > 0, x, alpha * x)

    def softmax(self, x):
        e_x = np.exp(x - np.max(x, axis=-1, keepdims=True))  # Numerical stability
        return e_x / np.sum(e_x, axis=-1, keepdims=True)

    def softplus(self, x):
        return np.log(1 + np.exp(x))

    # Loss function
    def compute_loss(self, y_true, y_pred):
        if self.loss_function == 'mse':
            return self.mean_squared_error(y_true, y_pred)
        elif self.loss_function == 'cross_entropy':
            return self.cross_entropy_loss(y_true, y_pred)
        else:
            raise ValueError(f"Loss function {self.loss_function} not supported.")

    # Mean Squared Error (MSE)
    def mean_squared_error(self, y_true, y_pred):
        return np.mean((y_true - y_pred) ** 2)

    # Cross-Entropy Loss (for classification)
    def cross_entropy_loss(self, y_true, y_pred):
        # To avoid log(0), we add a small epsilon value
        epsilon = 1e-15
        y_pred = np.clip(y_pred, epsilon, 1 - epsilon)
        return -np.mean(np.sum(y_true * np.log(y_pred), axis=1))

# Example usage:
```

```
input_size = 3    # Number of input neurons
hidden_size = 5   # Number of neurons in the hidden layer
output_size = 2   # Number of output neurons

# Create a neural network instance
nn = NeuralNetwork(input_size, hidden_size, output_size, activation_function='sigmoid',
loss_function='mse')

# Example input data (for a batch of 2 samples with 3 features each)
X_input = np.array([[0.1, 0.5, 0.9], [0.2, 0.8, 0.6]])

# Actual target values (e.g., for regression, use continuous values)
y_true = np.array([[0.5, 0.2], [0.7, 0.3]])

# Perform a forward pass with the input data
output = nn.forward(X_input)

# Calculate the loss using the true values and the predicted output
loss = nn.compute_loss(y_true, output)

print("\nPredicted output:")
print(output)
print("\nLoss (MSE):")
print(loss)
```

OUTPUT :

```
Predicted output:
[[0.87654603 0.21322158]
[0.79446709 0.24716415]]

Loss (MSE):
 0.03841934510105007
```

EXPERIMENT NO: 4

AIM : Perform an experiment to show how the choice of activation function effect the output of neuron experiment with logsig(n).

SOURCE CODE:

```
import numpy as np


class NeuralNetwork:
    def __init__(self, input_size, hidden_size, output_size, learning_rate=0.01):
        # Initialize the weights and biases
        self.weights_input_hidden = np.random.randn(input_size, hidden_size)
        self.bias_hidden = np.zeros((1, hidden_size))

        self.weights_hidden_output = np.random.randn(hidden_size, output_size)
        self.bias_output = np.zeros((1, output_size))

        # Learning rate for gradient descent
        self.learning_rate = learning_rate

    def forward(self, X):
        # Forward pass through the network
        self.hidden_input = np.dot(X, self.weights_input_hidden) + self.bias_hidden
        self.hidden_output = self.sigmoid(self.hidden_input)

        self.output_input = np.dot(self.hidden_output, self.weights_hidden_output) + self.bias_output
        self.output = self.sigmoid(self.output_input)

        return self.output

    def sigmoid(self, x):
        return 1 / (1 + np.exp(-x))

    def sigmoid_derivative(self, x):
```

```
        # Derivative of the sigmoid function
        return x * (1 - x)

    def compute_loss(self, y_true, y_pred):
        # Mean Squared Error (MSE) loss function
        return np.mean((y_true - y_pred) ** 2)

    def backward(self, X, y_true, y_pred):
        # Calculate the loss gradient with respect to the output layer
        output_error = y_pred - y_true
        output_delta = output_error * self.sigmoid_derivative(y_pred)

        # Calculate the loss gradient with respect to the hidden layer
        hidden_error = output_delta.dot(self.weights_hidden_output.T)
        hidden_delta = hidden_error * self.sigmoid_derivative(self.hidden_output)

        # Update weights and biases using the gradients
        self.weights_hidden_output -= self.learning_rate * self.hidden_output.T.dot(output_delta)
        self.bias_output -= self.learning_rate * np.sum(output_delta, axis=0, keepdims=True)

        self.weights_input_hidden -= self.learning_rate * X.T.dot(hidden_delta)
        self.bias_hidden -= self.learning_rate * np.sum(hidden_delta, axis=0, keepdims=True)

    def train(self, X, y_true, epochs=1000):
        # Train the neural network using forward and backward propagation
        for epoch in range(epochs):
            # Forward pass
            y_pred = self.forward(X)

            # Compute loss
            loss = self.compute_loss(y_true, y_pred)

            # Backward pass (backpropagation)
```

```
        self.backward(X, y_true, y_pred)

        # Print the loss at each epoch
        if epoch % 100 == 0:
            print(f"Epoch {epoch} | Loss: {loss}")

# Example usage:
input_size = 3    # Number of input neurons
hidden_size = 5   # Number of neurons in the hidden layer
output_size = 1   # Number of output neurons (for regression, it can be 1)

# Create the neural network instance
nn = NeuralNetwork(input_size, hidden_size, output_size, learning_rate=0.01)

# Example input data (for a batch of 4 samples with 3 features each)
X_input = np.array([[0.1, 0.5, 0.9],
                    [0.2, 0.8, 0.6],
                    [0.9, 0.1, 0.4],
                    [0.5, 0.3, 0.8]])

# Actual target values (e.g., for regression)
y_true = np.array([[0.5], [0.7], [0.2], [0.9]])

# Train the neural network
nn.train(X_input, y_true, epochs=1000)

# After training, let's test the network with the same input data
y_pred = nn.forward(X_input)

print("\nPredicted output after training:")
print(y_pred)
```

OUTPUT:

```
Epoch 0 | Loss: 0.07261206704099438
Epoch 100 | Loss: 0.06947570558090783
Epoch 200 | Loss: 0.06782901320496823
Epoch 300 | Loss: 0.06695239016938156
Epoch 400 | Loss: 0.06645255161349309
Epoch 500 | Loss: 0.06613132067718504
Epoch 600 | Loss: 0.0658932961768156
Epoch 700 | Loss: 0.06569374647874232
Epoch 800 | Loss: 0.0655120134274131
Epoch 900 | Loss: 0.06533863058660963
Predicted output after training:
[[0.5828925 ]
 [0.59719375]
[0.5631467 ]
[0.56629168]]
```

EXPERIMENT NO: 5

AIM: Perform an experiment to show how the perceptron learning rule works for linearly separable problems.

SOURCE CODE:

```
import numpy as np

class NeuralNetwork:
    def __init__(self, input_size, hidden_size, output_size, learning_rate=0.1):
        # Initialize weights and biases
        self.weights_input_hidden = np.random.randn(input_size, hidden_size)
        self.bias_hidden = np.zeros((1, hidden_size))

        self.weights_hidden_output = np.random.randn(hidden_size, output_size)
        self.bias_output = np.zeros((1, output_size))

        # Learning rate for gradient descent
        self.learning_rate = learning_rate

    def forward(self, X):
        # Forward pass: Calculate the activations for hidden and output layers
        self.hidden_input = np.dot(X, self.weights_input_hidden) + self.bias_hidden
        self.hidden_output = self.sigmoid(self.hidden_input)

        self.output_input = np.dot(self.hidden_output, self.weights_hidden_output) + self.bias_output
        self.output = self.sigmoid(self.output_input)

        return self.output

    def sigmoid(self, x):
        # Sigmoid activation function
        return 1 / (1 + np.exp(-x))

    def sigmoid_derivative(self, x):
        # Derivative of the sigmoid function
        return x * (1 - x)

    def compute_loss(self, y_true, y_pred):
        # Mean Squared Error loss function
        return np.mean((y_true - y_pred) ** 2)

    def backward(self, X, y_true, y_pred):
        # Backward pass: Calculate the gradients and update weights using gradient descent

        # Calculate output layer error and delta
```

```
        output_error = y_pred - y_true
        output_delta = output_error * self.sigmoid_derivative(y_pred)

        # Calculate hidden layer error and delta
        hidden_error = output_delta.dot(self.weights_hidden_output.T)
        hidden_delta = hidden_error * self.sigmoid_derivative(self.hidden_output)

        # Update weights and biases using the gradients
        self.weights_hidden_output -= self.learning_rate * self.hidden_output.T.dot(output_delta)
        self.bias_output -= self.learning_rate * np.sum(output_delta, axis=0, keepdims=True)

        self.weights_input_hidden -= self.learning_rate * X.T.dot(hidden_delta)
        self.bias_hidden -= self.learning_rate * np.sum(hidden_delta, axis=0, keepdims=True)

    def train(self, X, y_true, epochs=10000):
        # Train the neural network using forward and backward propagation
        for epoch in range(epochs):
            # Forward pass
            y_pred = self.forward(X)

            # Compute loss
            loss = self.compute_loss(y_true, y_pred)

            # Backward pass (backpropagation)
            self.backward(X, y_true, y_pred)

            # Print loss every 1000 epochs
            if epoch % 1000 == 0:
                print(f"Epoch {epoch} | Loss: {loss}")

    def test(self, X):
        # Test the neural network with the input data
        return self.forward(X)

# Example usage for the XOR problem:
# Input data for XOR (4 examples, 2 features each)
X_input = np.array([[0, 0],
                    [0, 1],
                    [1, 0],
                    [1, 1]])

# Output data for XOR (4 examples, 1 output each)
y_true = np.array([[0], [1], [1], [0]])

# Create a neural network instance
nn = NeuralNetwork(input_size=2, hidden_size=4, output_size=1, learning_rate=0.1)
```

```
# Train the neural network
print("Training the neural network...")
nn.train(X_input, y_true, epochs=10000)

# After training, test the neural network on the XOR input
print("\nTesting the neural network on the XOR input:")
y_pred = nn.test(X_input)

print("\nPredicted output after training:")
print(y_pred)
```

OUTPUT:

```
Training the neural network...
  Epoch 0 | Loss: 0.2526436661693838
  Epoch 1000 | Loss: 0.2397158667257264
  Epoch 2000 | Loss: 0.13062995557194468
  Epoch 3000 | Loss: 0.02872194228940983
  Epoch 4000 | Loss: 0.011393299508316048
  Epoch 5000 | Loss: 0.006560568046749035
  Epoch 6000 | Loss: 0.00447809366050457
  Epoch 7000 | Loss: 0.0033542019595272027
 Epoch 8000 | Loss: 0.002661533382957928
 Epoch 9000 | Loss: 0.00219593198121231

 Testing the neural network on the XOR input:

  Predicted output after training:
  [[0.02415875]
     [0.95222034]
   [0.95940909]
  [0.05421303]]
```

EXPERIMENT NO: 6

AIM : Perform an experiment to show how the perceptron learning rule works for Non-linearly separable problems.

Source code:

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv1D, MaxPooling1D, Flatten, Dense, Dropout
from tensorflow.keras.utils import to_categorical

# Load the dataset
file_path = "/content/sample_data/heart_disease.csv"  # Update this with the correct path
data = pd.read_csv(file_path)

# Assuming the target column is named "target" and features are numerical
X = data.drop(columns=["target"]).values
y = data["target"].values

# Preprocess the target variable
label_encoder = LabelEncoder()
y = label_encoder.fit_transform(y)
y = to_categorical(y)  # Convert to one-hot encoding for classification

# Preprocess the features
scaler = StandardScaler()
X = scaler.fit_transform(X)

# Reshape data for CNN input (CNN expects a 3D input: samples, timesteps, features)
X = np.expand_dims(X, axis=2)  # Add a dimension for features

# Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Build the CNN model
model = Sequential([
    Conv1D(filters=32, kernel_size=3, activation='relu', input_shape=(X.shape[1], 1)),
```

```
    MaxPooling1D(pool_size=2),
    Dropout(0.3),
    Conv1D(filters=64, kernel_size=3, activation='relu'),
    MaxPooling1D(pool_size=2),
    Dropout(0.3),
    Flatten(),
    Dense(128, activation='relu'),
    Dropout(0.3),
    Dense(y.shape[1], activation='softmax')  # Adjust for the number of classes
])

# Compile the model
model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])

# Train the model
history = model.fit(X_train, y_train, epochs=20, batch_size=32, validation_split=0.2,
verbose=1)

# Evaluate the model
test_loss, test_accuracy = model.evaluate(X_test, y_test, verbose=0)
print(f"Test Accuracy: {test_accuracy * 100:.2f}%")

# Save the model
model.save("heart_disease_cnn_model.h5")
```

OUTPUT:

```
  Epoch
- val_loss: 0.6508
  Epoch 10/20
                7/7 ——————————————————————0s 14ms/step - accuracy: 0.5485 - loss: 0.6771 - val_accuracy:
0.6939 - val_loss: 0.6536
Epoch 1/20
                7/7 ——————————————————————2s 57ms/step - accuracy: 0.3981 - loss: 0.7163 - val_accuracy:
0.5306 - val_loss: 0.6952
Epoch 2/20
                7/7 ——————————————————————0s 12ms/step - accuracy: 0.6080 - loss: 0.6850 - val_accuracy:
0.5918 - val_loss: 0.6914
                Epoch 3/20
                7/7 ——————————————————————0s 12ms/step - accuracy: 0.5316 - loss: 0.6904 - val_accuracy:
0.6327 - val_loss: 0.6823
                Epoch 4/20
                7/7 ——————————————————————0s 12ms/step - accuracy: 0.5459 - loss: 0.6858 - val_accuracy:
0.6531 - val_loss: 0.6750
                Epoch 5/20
                7/7 ——————————————————————0s 12ms/step - accuracy: 0.6064 - loss: 0.6799 - val_accuracy:
0.6327 - val_loss: 0.6690
                 Epoch 6/20
                7/7 ——————————————————————0s 14ms/step - accuracy: 0.5923 - loss: 0.6736 - val_accuracy:
```

0.6531 - val_loss: 0.6631

Epoch 7/20

7/7 ━━━━━━━━━━━━━━━━━━━━ **0s** 13ms/step - accuracy: 0.5802 - loss: 0.6710 - val_accuracy: 0.6327 - val_loss: 0.6576

Epoch 8/20

7/7 ━━━━━━━━━━━━━━━━━━━━ **0s** 12ms/step - accuracy: 0.6696 - loss: 0.6485 - val_accuracy: 0.6939 - val_loss: 0.6545

Epoch 9/20

7/7 ━━━━━━━━━━━━━━━━━━━━ **0s** 15ms/step - accuracy: 0.6508 - loss: 0.6444 - val_accuracy: 0.6939

Epoch 11/20

7/7 ━━━━━━━━━━━━━━━━━━━━ **0s** 14ms/step - accuracy: 0.7071 - loss: 0.6336 - val_accuracy: 0.6939 - val_loss: 0.6469

Epoch 12/20

7/7 ━━━━━━━━━━━━━━━━━━━━ **0s** 11ms/step - accuracy: 0.6538 - loss: 0.6388 - val_accuracy: 0.6939 - val_loss: 0.6394

Epoch 13/20

7/7 ━━━━━━━━━━━━━━━━━━━━ **0s** 12ms/step - accuracy: 0.6701 - loss: 0.6348 - val_accuracy: 0.6735 - val_loss: 0.6327

Epoch 14/20

7/7 ━━━━━━━━━━━━━━━━━━━━ **0s** 11ms/step - accuracy: 0.6995 - loss: 0.6027 - val_accuracy: 0.6735 - val_loss: 0.6265

Epoch 15/20

7/7 ━━━━━━━━━━━━━━━━━━━━ **0s** 7ms/step - accuracy: 0.6278 - loss: 0.6220 - val_accuracy: 0.6735 - val_loss: 0.6184

Epoch 16/20

7/7 ━━━━━━━━━━━━━━━━━━━━ **0s** 10ms/step - accuracy: 0.6609 - loss: 0.6122 - val_accuracy: 0.5918 - val_loss: 0.6339

Epoch 17/20

7/7 ━━━━━━━━━━━━━━━━━━━━ **0s** 9ms/step - accuracy: 0.7014 - loss: 0.5868 - val_accuracy: 0.5918 - val_loss: 0.6480

Epoch 18/20

7/7 ━━━━━━━━━━━━━━━━━━━━ **0s** 10ms/step - accuracy: 0.6149 - loss: 0.6221 - val_accuracy: 0.6939 - val_loss: 0.6028

Epoch 19/20

7/7 ━━━━━━━━━━━━━━━━━━━━ **0s** 7ms/step - accuracy: 0.6731 - loss: 0.5685 - val_accuracy: 0.6735 - val_loss: 0.5988

Epoch 20/20

7/7 ━━━━━━━━━━━━━━━━━━━━ **0s** 10ms/step - accuracy: 0.7165 - loss: 0.5732 - val_accuracy: 0.7143 - val_loss: 0.5899

Test Accuracy: 59.02%

EXPERIMENT :7

AIM : Write a program to draw a graph with multiple curve.

SOURCE CODE:

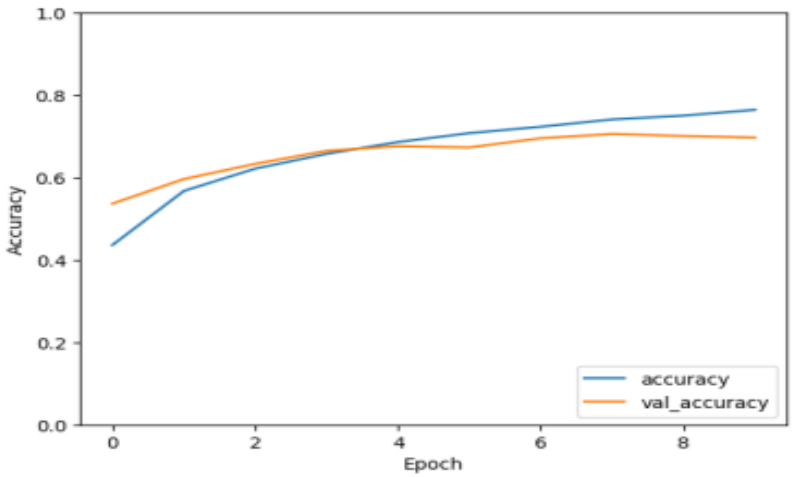
```
import tensorflow as tf
from tensorflow.keras import layers, models, datasets
import matplotlib.pyplot as plt

# Load and preprocess the dataset (using CIFAR-10 as an example)
def load_data():
    (x_train, y_train), (x_test, y_test) = datasets.cifar10.load_data()
    x_train, x_test = x_train / 255.0, x_test / 255.0  # Normalize to [0, 1]
    y_train = tf.keras.utils.to_categorical(y_train, 10)  # One-hot encoding
    y_test = tf.keras.utils.to_categorical(y_test, 10)
    return (x_train, y_train), (x_test, y_test)

# Define the CNN model
def create_model():
    model = models.Sequential()
    model.add(layers.Conv2D(32, (3, 3), activation='relu', input_shape=(32, 32, 3)))
    model.add(layers.MaxPooling2D((2, 2)))
    model.add(layers.Conv2D(64, (3, 3), activation='relu'))
    model.add(layers.MaxPooling2D((2, 2)))
    model.add(layers.Conv2D(64, (3, 3), activation='relu'))
    model.add(layers.Flatten())
    model.add(layers.Dense(64, activation='relu'))
    model.add(layers.Dense(10, activation='softmax'))  # Output layer for 10 classes
    return model

# Compile the model
def compile_model(model):
    model.compile(optimizer='adam',
                  loss='categorical_crossentropy',
                  metrics=['accuracy'])

# Train the model
def train_model(model, x_train, y_train, x_test, y_test):
    history = model.fit(x_train, y_train, epochs=10,
                        validation_data=(x_test, y_test), batch_size=64)
    return history

# Visualize training performance
def plot_history(history):
```

```
    plt.plot(history.history['accuracy'], label='accuracy')
    plt.plot(history.history['val_accuracy'], label='val_accuracy')
    plt.xlabel('Epoch')
    plt.ylabel('Accuracy')
    plt.ylim([0, 1])
    plt.legend(loc='lower right')
    plt.show()

# Main function
if __name__ == "__main__":
    # Load data
    (x_train, y_train), (x_test, y_test) = load_data()

    # Create the CNN model
    model = create_model()

    # Compile the model
    compile_model(model)

    # Train the model
    history = train_model(model, x_train, y_train, x_test, y_test)

    # Evaluate the model
    test_loss, test_acc = model.evaluate(x_test, y_test, verbose=2)
    print(f"Test accuracy: {test_acc:.2f}")

    # Plot training history
    plot_history(history)
```

OUTPUT:

```
Downloading data from https://www.cs.toronto.edu/~kriz/cifar-10-python.tar.gz
                170498071/170498071 ━━━━━━━━━━━━━━━━━━━━ 2s 0us/step
                 /usr/local/lib/python3.10/dist-packages/keras/src/layers/convolutional/base_conv.py:107:
UserWarning: Do not pass an
                 input_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an
`Input(shape)`
                object as the first layer in the model instead.
`                super().__init__(activity_regularizer=activity_regularizer, **kwargs)
                Epoch 1/10
              782/782 ━━━━━━━━━━━━━━━━━━━━ 70s 87ms/step - accuracy: 0.3473 - loss: 1.7702 -
val_accuracy: 0.5366 –
                val_loss: 1.2916
               Epoch 2/10
              782/782 ━━━━━━━━━━━━━━━━━━━━ 82s 87ms/step - accuracy: 0.5529 - loss: 1.2518 -
val_accuracy: 0.5964 -
               val_loss: 1.1262
               Epoch 3/10
               782/782 ━━━━━━━━━━━━━━━━━━━━ 85s 90ms/step - accuracy: 0.6156 - loss: 1.0826 -
val_accuracy: 0.6333 –
```

val_loss: 1.0362
Epoch 4/10
782/782 ━━━━━━━━━━━━━━━━━━━━ **69s** 89ms/step - accuracy: 0.6527 - loss: 0.9867 - val_accuracy: 0.6649 –
val_loss: 0.9496
Epoch 5/10
782/782 ━━━━━━━━━━━━━━━━━━━━ **85s** 92ms/step - accuracy: 0.6828 - loss: 0.9037 - val_accuracy: 0.6763 –
val_loss: 0.9290
Epoch 6/10
782/782 ━━━━━━━━━━━━━━━━━━━━ **79s** 89ms/step - accuracy: 0.7051 - loss: 0.8456 - val_accuracy: 0.6732
val_loss: 0.9262
Epoch 7/10
782/782 ━━━━━━━━━━━━━━━━━━━━ **80s** 87ms/step - accuracy: 0.7223 - loss: 0.7911 - val_accuracy: 0.6955
val_loss: 0.8803
Epoch 8/10
782/782 ━━━━━━━━━━━━━━━━━━━━ **83s** 88ms/step - accuracy: 0.7454 - loss: 0.7455 - val_accuracy: 0.7060
val_loss: 0.8540
Epoch 9/10
782/782 ━━━━━━━━━━━━━━━━━━━━ **81s** 87ms/step - accuracy: 0.7550 - loss: 0.7026 - val_accuracy: 0.7010
- val_loss: 0.8686
Epoch 10/10
782/782 ━━━━━━━━━━━━━━━━━━━━ **85s** 91ms/step - accuracy: 0.7669 - loss: 0.6697 - val_accuracy: 0.6972 -
val_loss: 0.8975
313/313 - 3s - 11ms/step - accuracy: 0.6972 - loss: 0.8975
Test accuracy: 0.70



EXPERIMENT :8

AIM: Setting up the Spyder IDE Environment and Executing a Python Program.

SOURCE CODE:

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense, Dropout
from tensorflow.keras.utils import to_categorical

# Load the dataset
file_path = "/content/sample_data/heart_disease.csv"  # Update this with your dataset's file path
data = pd.read_csv(file_path)

# Assuming the target column is named "target" and features are numerical
X = data.drop(columns=["target"]).values
y = data["target"].values

# Preprocess the target variable
label_encoder = LabelEncoder()
y = label_encoder.fit_transform(y)
y = to_categorical(y)  # Convert to one-hot encoding for classification
```

```
# Preprocess the features
scaler = StandardScaler()
X = scaler.fit_transform(X)

# Reshape data for RNN input (RNN expects a 3D input: samples, timesteps, features)
X = np.expand_dims(X, axis=1)  # Add a time dimension

# Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Build the RNN model
model = Sequential([
    LSTM(64, activation='tanh', return_sequences=True, input_shape=(X.shape[1], X.shape[2])),
    Dropout(0.3),
    LSTM(32, activation='tanh'),
    Dropout(0.3),
    Dense(128, activation='relu'),
    Dropout(0.3),
    Dense(y.shape[1], activation='softmax')  # Adjust for the number of classes
])

# Compile the model
model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])

# Train the model
```

```
history = model.fit(X_train, y_train, epochs=20, batch_size=32, validation_split=0.2, verbose=1)


# Evaluate the model

test_loss, test_accuracy = model.evaluate(X_test, y_test, verbose=0)

print(f"Test Accuracy: {test_accuracy * 100:.2f}%")


# Save the model

model.save("heart_disease_rnn_model.h5")
```

OUTPUT:

```
Epoch 1/20
/usr/local/lib/python3.10/dist-packages/keras/src/layers/rnn/rnn.py:204: UserWarning: Do not pass an `input_shape`/`input_dim`
argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(**kwargs)
7/7 ━━━━━━━━━━━━━━━━━━━━ 4s 89ms/step - accuracy: 0.6326 - loss: 0.6909 - val_accuracy: 0.7551 –
val_loss: 0.6797
Epoch 2/20
7/7 ━━━━━━━━━━━━━━━━━━━━ 1s 17ms/step - accuracy: 0.6951 - loss: 0.6819 - val_accuracy: 0.7347 –
val_loss: 0.6672
Epoch 3/20
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - accuracy: 0.7393 - loss: 0.6709 - val_accuracy: 0.8367 -
val_loss: 0.6516
Epoch 4/20
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - accuracy: 0.8300 - loss: 0.6518 - val_accuracy: 0.8367 -
val_loss: 0.6336
Epoch 5/20
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.8382 - loss: 0.6336 - val_accuracy: 0.8571 –
val_loss: 0.6110
Epoch 6/20
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step - accuracy: 0.8269 - loss: 0.6146 - val_accuracy: 0.8571 –
val_loss: 0.5793
Epoch 7/20
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - accuracy: 0.8707 - loss: 0.5787 - val_accuracy: 0.8571 –
val_loss: 0.5366
Epoch 8/20
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - accuracy: 0.8612 - loss: 0.5197 - val_accuracy: 0.8571 –
```

val_loss: 0.4886
Epoch 9/20
7/7 ━━━━━━━━━━━━━━━━━━━━ **0s** 14ms/step - accuracy: 0.8397 - loss: 0.4870 - val_accuracy: 0.8571 –
val_loss: 0.4586

Epoch 10/20
7/7 ━━━━━━━━━━━━━━━━━━━━ **0s** 17ms/step - accuracy: 0.8460 - loss: 0.4421 - val_accuracy: 0.8367 –
val_loss: 0.4361
Epoch 11/20
7/7 ━━━━━━━━━━━━━━━━━━━━ **0s** 25ms/step - accuracy: 0.8611 - loss: 0.4175 - val_accuracy: 0.8571 –
val_loss: 0.4101
Epoch 12/20
7/7 ━━━━━━━━━━━━━━━━━━━━ **0s** 16ms/step - accuracy: 0.8663 - loss: 0.3781 - val_accuracy: 0.8571 –
val_loss: 0.3930
Epoch 13/20
7/7 ━━━━━━━━━━━━━━━━━━━━ **0s** 14ms/step - accuracy: 0.8723 - loss: 0.3492 - val_accuracy: 0.8571 –
val_loss: 0.3814
Epoch 14/20
7/7 ━━━━━━━━━━━━━━━━━━━━ **0s** 14ms/step - accuracy: 0.8477 - loss: 0.4125 - val_accuracy: 0.8571 –
val_loss: 0.3775
Epoch 15/20
7/7 ━━━━━━━━━━━━━━━━━━━━ **0s** 9ms/step - accuracy: 0.8768 - loss: 0.3225 - val_accuracy: 0.8367 –
val_loss: 0.3859
Epoch 16/20
7/7 ━━━━━━━━━━━━━━━━━━━━ **0s** 14ms/step - accuracy: 0.8781 - loss: 0.3256 - val_accuracy: 0.8163 –
val_loss: 0.3909
Epoch 17/20
7/7 ━━━━━━━━━━━━━━━━━━━━ **0s** 11ms/step - accuracy: 0.8708 - loss: 0.3154 - val_accuracy: 0.8367 –
val_loss: 0.3893
Epoch 18/20
7/7 ━━━━━━━━━━━━━━━━━━━━ **0s** 9ms/step - accuracy: 0.8408 - loss: 0.3696 - val_accuracy: 0.8571 –
val_loss: 0.3845
Epoch 19/20
7/7 ━━━━━━━━━━━━━━━━━━━━ **0s** 11ms/step - accuracy: 0.8565 - loss: 0.3285 - val_accuracy: 0.8571 –
val_loss: 0.3792
Epoch 20/20
7/7 ━━━━━━━━━━━━━━━━━━━━ **0s** 9ms/step - accuracy: 0.8575 - loss: 0.3406 - val_accuracy: 0.8571 –
val_loss: 0.3810
Test Accuracy: 85.25%

EXPERIMENT 9:

AIM : Installing Keras, Tensorflow and Pytorch libraries and making use of them.

SOURCE CODE:

```
import cv2
import matplotlib.pyplot as plt


# Load pre-trained Haar Cascade classifier for face detection
haarcascade_path = cv2.data.haarcascades + 'haarcascade_frontalface_default.xml'
face_cascade = cv2.CascadeClassifier(haarcascade_path)


 def detect_faces(image_path):
    """Detect faces in an image using Haar Cascade."""
    # Load the image
    image = cv2.imread(image_path)
    gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)  # Convert to grayscale

    # Detect faces
    faces = face_cascade.detectMultiScale(gray, scaleFactor=1.1, minNeighbors=5, minSize=(30, 30))

    # Draw rectangles around detected faces
    for (x, y, w, h) in faces:
        cv2.rectangle(image, (x, y), (x + w, y + h), (255, 0, 0), 2)

    # Convert the image to RGB for displaying
```

```
    image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

    # Display the image
    plt.imshow(image_rgb)
    plt.axis("off")
    plt.show()

    print(f"Detected {len(faces)} face(s).")

# Test the program with an example image
image_path = "/content/sample_data/IMG-20240802-WA0134.jpg"  # Replace with the path to your image
detect_faces(image_path)
```

OUTPUT:

EXPERIMENT 10:

AIM: Train a sentiment analysis model on IMDB dataset, use RNN layers with LSTM/GRU notes.

SOURCE CODE:

```
import cv2
import matplotlib.pyplot as plt

# Load pre-trained Haar Cascade classifier for face detection
haarcascade_path = cv2.data.haarcascades + 'haarcascade_frontalface_default.xml'
face_cascade = cv2.CascadeClassifier(haarcascade_path)

def detect_faces(image_path):
    """Detect faces in an image using Haar Cascade."""
    # Load the image
    image = cv2.imread(image_path)
    gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)  # Convert to grayscale

    # Detect faces
    faces = face_cascade.detectMultiScale(gray, scaleFactor=1.1, minNeighbors=5, minSize=(30, 30))

    # Draw rectangles around detected faces
    for (x, y, w, h) in faces:
        cv2.rectangle(image, (x, y), (x + w, y + h), (255, 0, 0), 2)

    # Convert the image to RGB for displaying
```

```
    image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)


    # Display the image
    plt.imshow(image_rgb)
    plt.axis("off")
    plt.show()


    print(f"Detected {len(faces)} face(s).")


# Test the program with an example image
image_path = "/content/sample_data/IMG-20240802-WA0134.jpg"  # Replace with the path to your image
detect_faces(image_path)
```

OUTPUT:

1/1 ━━━━━━━━━━━━━━━━━━━━ **1s** 1s/step

Predictions:
1: Band_Aid (97.77%)
2: spatula (1.03%)
3: rubber_eraser (0.07%)
4: Windsor_tie (0.04%)
5: letter_opener (0.04%)

EXPERIMENT 11:

AIM: Applying the Autoencoder algorithms for encoding the real-world data.

SOURCE CODE:

```
import numpy as np
import tensorflow as tf
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Input, Dense
from tensorflow.keras.datasets import mnist
import matplotlib.pyplot as plt


# Load dataset (you can replace MNIST with your own dataset)
(x_train, _), (x_test, _) = mnist.load_data()


# Normalize data
x_train = x_train.astype("float32") / 255.0
x_test = x_test.astype("float32") / 255.0


# Flatten images
x_train = x_train.reshape((x_train.shape[0], -1))  # 28x28 -> 784
x_test = x_test.reshape((x_test.shape[0], -1))


input_dim = x_train.shape[1]  # 784


# Define the autoencoder
```

```
encoding_dim = 64  # Number of neurons in the bottleneck layer

# Encoder
input_layer = Input(shape=(input_dim,))
encoded = Dense(128, activation="relu")(input_layer)
encoded = Dense(encoding_dim, activation="relu")(encoded)

# Decoder
decoded = Dense(128, activation="relu")(encoded)
decoded = Dense(input_dim, activation="sigmoid")(decoded)

# Autoencoder model
autoencoder = Model(input_layer, decoded)

# Encoder model for encoding data
encoder = Model(input_layer, encoded)

# Compile the autoencoder
autoencoder.compile(optimizer="adam", loss="binary_crossentropy")

# Train the autoencoder
history = autoencoder.fit(
    x_train,
    x_train,
    epochs=20,
```

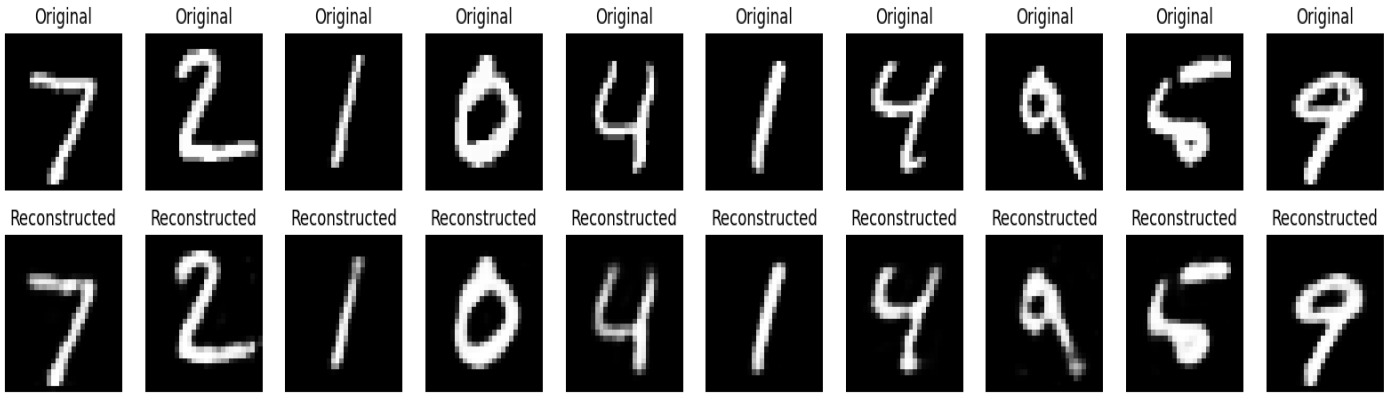
```
    batch_size=256,
    shuffle=True,
    validation_data=(x_test, x_test),
)


# Visualize reconstruction performance
def visualize_reconstruction(autoencoder, x_test, n=10):
    decoded_imgs = autoencoder.predict(x_test[:n])
    plt.figure(figsize=(20, 4))

    for i in range(n):
        # Original images
        ax = plt.subplot(2, n, i + 1)
        plt.imshow(x_test[i].reshape(28, 28), cmap="gray")
        plt.title("Original")
        plt.axis("off")

        # Reconstructed images
        ax = plt.subplot(2, n, i + 1 + n)
        plt.imshow(decoded_imgs[i].reshape(28, 28), cmap="gray")
        plt.title("Reconstructed")
        plt.axis("off")

    plt.show()
```

```
visualize_reconstruction(autoencoder, x_test)


# Encode real-world data

encoded_data = encoder.predict(x_test)


print(f"Shape of encoded data: {encoded_data.shape}")
```

OUTPUT:

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11490434/11490434 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Epoch 1/20
235/235 ━━━━━━━━━━━━━━━━━━━━ 6s 18ms/step - loss: 0.3288 - val_loss: 0.1394
Epoch 2/20
235/235 ━━━━━━━━━━━━━━━━━━━━ 3s 14ms/step - loss: 0.1316 - val_loss: 0.1114
Epoch 3/20
235/235 ━━━━━━━━━━━━━━━━━━━━ 5s 16ms/step - loss: 0.1101 - val_loss: 0.1015
Epoch 4/20
235/235 ━━━━━━━━━━━━━━━━━━━━ 4s 18ms/step - loss: 0.1017 - val_loss: 0.0964
Epoch 5/20
235/235 ━━━━━━━━━━━━━━━━━━━━ 3s 14ms/step - loss: 0.0964 - val_loss: 0.0930
Epoch 6/20
235/235 ━━━━━━━━━━━━━━━━━━━━ 3s 14ms/step - loss: 0.0920 - val_loss: 0.0887
Epoch 7/20
235/235 ━━━━━━━━━━━━━━━━━━━━ 4s 18ms/step - loss: 0.0891 - val_loss: 0.0870
Epoch 8/20
235/235 ━━━━━━━━━━━━━━━━━━━━ 4s 15ms/step - loss: 0.0870 - val_loss: 0.0847
Epoch 9/20
235/235 ━━━━━━━━━━━━━━━━━━━━ 5s 15ms/step - loss: 0.0854 - val_loss: 0.0834
Epoch 10/20
235/235 ━━━━━━━━━━━━━━━━━━━━ 4s 16ms/step - loss: 0.0839 - val_loss: 0.0821
Epoch 11/20
235/235 ━━━━━━━━━━━━━━━━━━━━ 4s 17ms/step - loss: 0.0828 - val_loss: 0.0817
Epoch 12/20
235/235 ━━━━━━━━━━━━━━━━━━━━ 4s 14ms/step - loss: 0.0818 - val_loss: 0.0804
Epoch 13/20
235/235 ━━━━━━━━━━━━━━━━━━━━ 3s 13ms/step - loss: 0.0811 - val_loss: 0.0798
Epoch 14/20
235/235 ━━━━━━━━━━━━━━━━━━━━ 6s 15ms/step - loss: 0.0804 - val_loss: 0.0791


Epoch 15/20
235/235 ━━━━━━━━━━━━━━━━━━━━ 3s 14ms/step - loss: 0.0798 - val_loss: 0.0787
Epoch 16/20
```

235/235 ━━━━━━━━━━━━━━━━━━━━ **6s** 18ms/step - loss: 0.0793 - val_loss: 0.0783
Epoch 17/20
235/235 ━━━━━━━━━━━━━━━━━━━━ **4s** 14ms/step - loss: 0.0790 - val_loss: 0.0779
Epoch 18/20
235/235 ━━━━━━━━━━━━━━━━━━━━ **3s** 14ms/step - loss: 0.0785 - val_loss: 0.0777
Epoch 19/20
235/235 ━━━━━━━━━━━━━━━━━━━━ **3s** 14ms/step - loss: 0.0782 - val_loss: 0.0771
Epoch 20/20
235/235 ━━━━━━━━━━━━━━━━━━━━ **5s** 14ms/step - loss: 0.0779 - val_loss: 0.0770



EXPERIMENT 12:

AIM: Applying Generative Adversial Networks for image generation and unsupervised tasks**.**

SOURCE CODE:

```
import tensorflow as tf
from tensorflow.keras import layers
import numpy as np
import matplotlib.pyplot as plt


# Load MNIST dataset
(X_train, _), (_, _) = tf.keras.datasets.mnist.load_data()
X_train = X_train / 127.5 - 1.0  # Normalize to [-1, 1]
X_train = np.expand_dims(X_train, axis=-1)  # Add channel dimension


# Parameters
latent_dim = 100
img_shape = X_train.shape[1:]


# Generator
def build_generator():
    model = tf.keras.Sequential([
        layers.Dense(128, activation="relu", input_dim=latent_dim),
        layers.BatchNormalization(),
        layers.Dense(256, activation="relu"),
        layers.BatchNormalization(),
```

```
        layers.Dense(512, activation="relu"),
        layers.BatchNormalization(),
        layers.Dense(np.prod(img_shape), activation="tanh"),
        layers.Reshape(img_shape)
    ])
    return model

# Discriminator
def build_discriminator():
    model = tf.keras.Sequential([
        layers.Flatten(input_shape=img_shape),
        layers.Dense(512, activation="relu"),
        layers.Dense(256, activation="relu"),
        layers.Dense(1, activation="sigmoid")
    ])
    return model

# Build and compile models
generator = build_generator()
discriminator = build_discriminator()
discriminator.compile(optimizer=tf.keras.optimizers.Adam(0.0002, 0.5), loss="binary_crossentropy",
metrics=["accuracy"])

# GAN model
discriminator.trainable = False
```

```
gan_input = layers.Input(shape=(latent_dim,))
generated_img = generator(gan_input)
validity = discriminator(generated_img)
gan = tf.keras.Model(gan_input, validity)
gan.compile(optimizer=tf.keras.optimizers.Adam(0.0002, 0.5), loss="binary_crossentropy")

# Training function
def train_gan(epochs, batch_size, save_interval):
    valid = np.ones((batch_size, 1))
    fake = np.zeros((batch_size, 1))

    for epoch in range(epochs):
        # Train Discriminator
        idx = np.random.randint(0, X_train.shape[0], batch_size)
        real_imgs = X_train[idx]

        noise = np.random.normal(0, 1, (batch_size, latent_dim))
        fake_imgs = generator.predict(noise)

        d_loss_real = discriminator.train_on_batch(real_imgs, valid)
        d_loss_fake = discriminator.train_on_batch(fake_imgs, fake)
        d_loss = 0.5 * np.add(d_loss_real, d_loss_fake)

        # Train Generator
        noise = np.random.normal(0, 1, (batch_size, latent_dim))
```

```
        g_loss = gan.train_on_batch(noise, valid)


        # Print progress
        print(f"{epoch + 1}/{epochs} [D loss: {d_loss[0]} | D acc: {100 * d_loss[1]:.2f}%] [G loss: {g_loss}]")


        # Save generated images
        if epoch % save_interval == 0:
            save_generated_images(epoch)


# Save generated images
def save_generated_images(epoch, examples=25, dim=(5, 5), figsize=(10, 10)):
    noise = np.random.normal(0, 1, (examples, latent_dim))
    generated_imgs = generator.predict(noise)
    generated_imgs = 0.5 * generated_imgs + 0.5  # Rescale to [0, 1]


    plt.figure(figsize=figsize)
    for i in range(examples):
        plt.subplot(dim[0], dim[1], i + 1)
        plt.imshow(generated_imgs[i, :, :, 0], cmap="gray")
        plt.axis("off")
    plt.tight_layout()
    plt.savefig(f"generated_{epoch}.png")
    plt.close()
                # Train GAN
```

train_gan(epochs=500, batch_size=64, save_interval=1000)

OUTPUT:

2/2 ━━━━━━━━━━━━━━━━━━━━ **0s** 4ms/step
3/500 [D loss: 0.8219563961029053 | D acc: 47.01%] [G loss: [array(0.79799986, dtype=float32), array(0.79799986,
dtype=float32), array(0.5182292, dtype=float32)]]
2/2 ━━━━━━━━━━━━━━━━━━━━ **0s** 3ms/step
4/500 [D loss: 0.819179892539978 | D acc: 48.31%] [G loss: [array(0.8021188, dtype=float32), array(0.8021188, dtype=float32),
array(0.5175781, dtype=float32)]]
2/2 ━━━━━━━━━━━━━━━━━━━━ **0s** 4ms/step
5/500 [D loss: 0.8161396980285645 | D acc: 49.04%] [G loss: [array(0.80366504, dtype=float32), array(0.80366504,
dtype=float32), array(0.5171875, dtype=float32)]]
2/2 ━━━━━━━━━━━━━━━━━━━━ **0s** 5ms/step
6/500 [D loss: 0.8108803629875183 | D acc: 49.77%] [G loss: [array(0.80106044, dtype=float32), array(0.80106044,
dtype=float32), array(0.51953125, dtype=float32)]]
2/2 ━━━━━━━━━━━━━━━━━━━━ **0s** 7ms/step
7/500 [D loss: 0.8115783929824829 | D acc: 49.76%] [G loss: [array(0.80331516, dtype=float32), array(0.80331516,
dtype=float32), array(0.515625, dtype=float32)]]
2/2 ━━━━━━━━━━━━━━━━━━━━ **0s** 4ms/step
8/500 [D loss: 0.813327431678772 | D acc: 50.15%] [G loss: [array(0.8066977, dtype=float32), array(0.8066977, dtype=float32),
array(0.5175781, dtype=float32)]]
2/2 ━━━━━━━━━━━━━━━━━━━━ **0s** 4ms/step
9/500 [D loss: 0.8112155199050903 | D acc: 50.32%] [G loss: [array(0.80519795, dtype=float32), array(0.80519795,
dtype=float32), array(0.5173611, dtype=float32)]]
2/2 ━━━━━━━━━━━━━━━━━━━━ **0s** 4ms/step
10/500 [D loss: 0.8127102851867676 | D acc: 50.49%] [G loss: [array(0.8075258, dtype=float32), array(0.8075258, dtype=float32),
array(0.5171875, dtype=float32